Proving Contract Assertions Are Not Violated

Document #:	D4405R0
Date:	2026-10-29
Project:	Programming Language C++
Audience:	EWG

Reply-to:	Joshua Berne <jberne4@bloomberg.net>
	Jonathan Grant <jgrantonline@gmail.com>

Abstract

Integrating contract assertions with static analysis to prove that they are never violated has long been an intended goal for C++26 Contracts to evolve to more fully support. There are various ways to model this desire and support it more fully in the Standard, with different tradeoffs, and in this paper we explore alternatives for this and propose two new evaluation semantics to capture the desire — *prove* and *diagnose*. These semantics are applied in an additional *analysis evaluation* of each contract assertion, selected independently of its runtime evaluations, and we propose additions to P3400 and P3595 to control it.

Contents

1 Introduction **3**
- 1.1 Local Proofs 4
- 1.2 Proving Preconditions 5
- 1.3 Virtual Functions 6

2 Proposal **7**
- 2.1 New Evaluation Semantics 8
- 2.2 The Analysis Evaluation 9
  - 2.2.1 Controlling the Analysis Semantic with Labels 11
- 2.3 Analysis Semantic Configuration 12
  - 2.3.1 Adjusting to the Allowed Semantics 14
- 2.4 Other Interactions 15

3 Discussion **16**
- 3.1 Design of the Semantics 16
- 3.2 Virtual Functions 17
- 3.3 Premises and Assumptions 18
- 3.4 Portability 19

4 Implementation Experience **19**
- 4.1 The Semantics and the Analysis Evaluation 19
- 4.2 Taking the Address of a Function 21
- 4.3 Configuration 23

4.4 Proving Strategies 23
4.5 Previous Assertions and Undefined Behavior 26
4.6 Assuming What Was Proven 27
4.7 Cost 28

5 Wording Changes 29

6 Conclusion 31

Revision History

Revision 0

- Original version of the paper

1 Introduction

Numerous papers have come to WG21 ([P4019R1], [P4021R2], and [P4363R0]) proposing various forms of a construct that provides a predicate expression which the compiler must prove always evaluates to `true` if the resulting program is well-formed.

- Unlike `static_assert`, these expressions can involve non-constant values.
- Unlike a contract assertion with the current evaluation semantics, a potential violation must make the program ill-formed (or at a minimum issue a diagnostic).
- Like a contract assertion with the *ignore* (or *assume*) semantic, the construct does not evaluate the predicate as part of evaluating the enclosing code.

There are pros and cons to the feature in general, such as whether the compiler's ability to prove any given expression will be stable across optimization levels or compiler versions. Our goal in this paper is not to fully explore those tradeoffs, but to explore how the facility could be provided as part of C++26 Contracts (including how that might integrate with the other papers being proposed for C++29 as enumerated in [P3850R1]).

Other than tautological cases where a predicate is a constant expression, identifying whether a contract assertion can be violated depends on data and control flow analysis that compilers do already at many levels, but whose results can vary greatly with QoI and many other factors. Therefore, we need to express that certain contract assertions, evaluated in certain contexts, are those that we expect to see proven true. That expression can be achieved by saying that a particular evaluation of a contract assertion happens with one of two new semantics:

- `prove`: A contract assertion is *proved* (or *proven*) if the compiler can guarantee that that evaluation of the assertion would never be a runtime violation if checked. If such a contract assertion evaluation's result cannot be guaranteed, the program is ill-formed (and an associated diagnostic is provided).
- `diagnose`: As a weaker condition, we can ask a compiler to attempt to *prove* a given contract assertion evaluation and simply inform us (by emitting a diagnostic message) when that proof cannot be achieved. Such an evaluation would otherwise do nothing at runtime — although the assertion can always be evaluated again at runtime with one of the existing evaluation semantics.

On their own, these two new semantics are simply new evaluation semantics with which a contract assertion can be evaluated. Where they are most useful, however, is not in place of the existing semantics but alongside them — we often want to be warned at compile time that a precondition might be violated *and* still have it enforced at runtime if it is. The C++26 Contracts model already allows a contract assertion to be evaluated more than once, so we recommend that implementations

give each contract assertion an additional *analysis evaluation*, which is always its first evaluation, and which is the only evaluation to which *prove* or *diagnose* is ever applied:

- For a precondition, the analysis evaluation is at each call, in the caller, where more information about the range of possible values for the arguments is known.
- For a postcondition or a `contract_assert`, the analysis evaluation is in the function body, where the code that must establish it is visible.

The semantic of the analysis evaluation is one of *ignore*, *diagnose*, or *prove*, and it is selected independently of the semantics of the assertion's other, runtime, evaluations — with a configuration system such as that proposed by [P3595R1] (Section 2.3) or with the assertion-control objects of [P3400R5] (Section 2.2.1).

The rest of this section shows what that makes possible: proofs within a function body (Section 1.1), proofs of preconditions at each call (Section 1.2), and proofs that the overriders of a virtual function honor the contracts of the functions they override (Section 1.3). Section 2 then describes the proposed changes that achieve this goal, both on top of C++26 Contracts, assertion-control objects from [P3400R5], or the command-line configuration system described in [P3595R1].

1.1 Local Proofs

By combining the new semantics with assertion-control objects (as introduced in [P3400R5]) we are able to require that a particular assertion be proven in all cases, which could be added to the Standard Library as a new label `std::contracts::labels::always_prove`:

```
namespace std::contracts::labels {
struct always_prove_t {
  using assertion_control_object = always_prove_t;
  static constexpr evaluation_semantic_set
    allowed_semantics = {evaluation_semantic::prove};
};
constexpr always_prove_t always_prove{};
}
```

Since *prove* is a semantic only for the analysis evaluation, this label allows no runtime semantic at all. Whatever the build configures, the analysis evaluation of an assertion with this label is therefore *prove*, and its runtime evaluations are *ignore* — a proof makes them redundant (see Section 2.3.1). A label `always_diagnose` is defined the same way, allowing only *diagnose*.

Because the analysis evaluation of a precondition happens at each call, a precondition with this label must be proven at every call:

Try this: GCC Clang

```
void f(int x) pre<always_prove>(x > 0);

void g(int y)
{
  f(1);         // OK, proven
  f(y);         // Error, cannot be proven
  if (y > 0) {
    f(y);       // OK, proven
```

```
  }
}
```

A call through a pointer to `f` would have no analysis evaluation, and `f` itself never checks its precondition, so for such calls the precondition could be proven only if it were tautological, which `x > 0` is not (see Section 2.2).

Within a function body on a `contract_assert`, we can take advantage of these labels to require that the assertion be proven true by the compiler, or to be told when it is not:

Try this: GCC Clang

```
void f(int x)
{
  contract_assert<always_diagnose>(x > 0);  // OK, warns: cannot be proven
  if (x > 0) {
    contract_assert<always_prove>(x > 0);   // OK, proven
  }
}
```

Neither of these assertions is ever checked at runtime. To be told when an assertion cannot be proven while leaving its runtime evaluations to the build configuration, a label can instead set only the semantic of the analysis evaluation, as shown in Section 2.2.1.

We can similarly make that requirement in a postcondition and have it checked against the function body:

Try this: GCC Clang

```
int g(bool b)
  post<always_prove>(r : r > 0)  // OK, proven
{
  return b ? 17 : 34;
}
```

1.2 Proving Preconditions

Preconditions cannot, in general, be provably true for a function (and those that can are not particularly useful). What can be done, however, is to require that a contract assertion be provable at each call, which is exactly where the analysis evaluation of a precondition takes place. We can use a [P3400R5] control object to put those preconditions we want to prove in all callers in a group that we can control:

Try this: GCC Clang

```
void f(int x) pre<"required"group>(x > 0);

void g()
{
  f(1);  // OK
  f(0);  // Error if the "required" group has the prove semantic
}
```

Then we can use a contract configuration (as proposed in [P3595R1], with the `analysis` key we propose in Section 2.3) to give the analysis evaluations of that group's assertions the *prove* semantic:

```
[
  {"match": {"group": "required", "analysis": true},
   "output": {"semantic": "prove"}}
]
```

This entry says nothing about the runtime evaluations of the precondition, so it is checked (or not) exactly as it would be without the entry.

With this configuration both of our implementations prove `f(1)` and reject `f(0)`, whose predicate is always false once the argument is substituted for the parameter, while every other contract assertion keeps the semantics it would otherwise have. A call `f(y)` would be an error as well, unless the caller had established `y > 0` (by testing it first, for example). Section 4.3 shows a broader configuration of the same kind.

1.3 Virtual Functions

With support for contract assertions on virtual functions as proposed in [P3097R3] (approved for C++29), a call through a reference or pointer to a base class evaluates the contract assertions of the function it names, which the caller can see, along with those of the final overrider, which it cannot. The caller can prove the preconditions of the function it names, as at any other call, but whether the final overrider honors that contract can be analyzed only where the overrider is defined. Consider two unrelated base classes whose functions `f` accept, and return, values from two ranges that do not overlap, and a class that overrides both:

Try this: GCC Clang

```
struct B1 {
  virtual int f(int x)
    pre(x >= 0 && x <= 10)
    post(r: r >= 0 && r <= 10);
};

struct B2 {
  virtual int f(int x)
    pre(x >= 20 && x <= 30)
    post(r: r >= 20 && r <= 30);
};

struct D : B1, B2 {
  int f(const int x) override
    pre(x >= 0 && x <= 30)  // OK, proven from either base's precondition
    post(r: r == x)         // OK, proven
  {
    return x;  // OK, the postconditions of B1::f and B2::f are proven
  }
};

struct E : B1 {
```

```
  int f(int x) override
    pre(x >= 5 && x <= 10)  // Error, not implied by B1::f's precondition
  {
    return x;
  }
};

void g(B1& b, int y)
{
  b.f(5);         // OK, proven
  b.f(y);         // Error, cannot be proven
  if (y >= 0 && y <= 10) {
    b.f(y);       // OK, proven
  }
}
```

With every analysis evaluation given the *prove* semantic:

- In `g`, the analysis evaluation of the precondition of `B1::f` is made at each call through `b`, just as at any other call: `b.f(5)` and the guarded call are proven, and `b.f(y)` is not.
- A call through `B1` that reaches `D::f` has established that `x` is between 0 and 10. The precondition of `D::f` then holds, and `D::f` returns `x`, which satisfies both its own postcondition and that of `B1::f`. The same reasoning, with `x` between 20 and 30, covers a call through `B2`. Neither proof needs anything from the other base, even though no argument satisfies the preconditions of both, and no result the postconditions of both.
- A call through `B1` that reaches `E::f` might pass any value from 0 to 4, which violates the precondition of `E::f`, so that precondition cannot be proven and the program is ill-formed. The error is reported where `E::f` is defined, since that is the only place where the contract of `E::f` can be compared with that of `B1::f`.

To make these proofs, the body of each overrider is analyzed once more for each function it overrides, taking that function's preconditions as given (Section 2.2). Both of our implementations behave as the comments say. That analysis has a cost, and with `-fno-contracts-prove-override` both instead require the preconditions of `D::f` to be tautological for a call through `B1` or `B2`, and `D::f` to establish the postconditions of `B1::f` and `B2::f` with nothing given. Since `x` could then be anything, neither the precondition of `D::f` nor the postconditions of the bases can be proven, and the program is rejected at `D::f` as well (see Section 3.2).

2 Proposal

Our proposal has several pieces, and they do not all change the same document. Each falls into exactly one of four categories:

- *Proposed change to the Standard*: the *prove* and *diagnose* evaluation semantics, including how they behave during constant evaluation (Sections 2.1 and 2.2). These are what our wording covers.

- *Proposed additions on top of [P3400R5]*: the `analysis_semantic` facet of assertion-control objects (Section 2.2.1), and the allowed sets of the `terminating` label, extended with *prove*, and of the `symbolic` label, extended with *prove* and *diagnose* (Section 2.4). Wording for this feature will be provided relative to [P3400R5] in a future revision.
- *Proposed additions to [P3595R1]*: the `"analysis"` match key and its rules, a command-line flag that sets the analysis semantic (Section 2.3), and the split allowed sets and the resolution of the semantics of each evaluation (Section 2.3.1).
- We then explore some implications of our design and clarify what follows from the proposals above in the context of C++26 Contracts.

2.1 New Evaluation Semantics

The two semantics differ only in what happens when the compiler cannot show that an evaluation never results in a contract violation. With *prove*, the program is then ill-formed; with *diagnose*, the compiler issues a diagnostic and the program is otherwise unaffected. Neither evaluates the predicate, and neither does anything at runtime, so an evaluation with either semantic changes nothing about what the program does — only whether, and with what diagnostics, it compiles.

What can be proven about an evaluation depends on where it takes place. A function contract assertion can be evaluated in the caller or in the callee — the C++26 Contracts model leaves it to the implementation whether a precondition is checked on the caller side or the callee side — and the two know very different things. At a call, the caller knows the arguments and whatever it has established about them; in the callee, a precondition is a statement about every argument that any caller might pass. Requiring a proof where there is little information inevitably leads to not being able to prove that the violation will not happen: a precondition evaluated with *prove* in the callee can be proven only when it is always true, whatever the arguments, and a postcondition evaluated with *prove* in a caller that cannot see the function body fares no better. The semantics themselves do not choose where they are applied; that is the subject of Section 2.2.

Proposal (Standard): Add the *prove* evaluation semantic

Introduce a new contract evaluation semantic, *prove*.
When a contract assertion is evaluated with the *prove* semantic, a program is ill-formed if that evaluation can result in a contract violation. During constant evaluation, *prove* is treated as *enforce* (Section 2.2).

Proposal (Standard): Add the *diagnose* evaluation semantic

Introduce a new contract evaluation semantic, *diagnose*.
When a contract assertion is evaluated with the *diagnose* semantic, issue a diagnostic if that evaluation can result in a contract violation. During constant evaluation, *diagnose* is treated as *observe* (Section 2.2).

2.2 The Analysis Evaluation

What we introduce here is an *analysis evaluation*: an evaluation of each contract assertion that the compiler inserts, in addition to the evaluations the assertion already has. The C++26 Contracts model already allows this. An evaluation of a contract assertion may be repeated an implementation-defined number of times, the evaluation semantic of each evaluation is implementation-defined, and the semantics can differ for different evaluations of the same contract assertion ([basic.contract.eval]); the model also leaves it to the implementation whether a precondition is evaluated on the caller side or the callee side. An implementation can therefore insert as many evaluations of a contract assertion as it wants and give them whatever semantics it chooses. We leverage that flexibility to give *prove* and *diagnose* a place to be applied where the most information is available for any given assertion.

We need such a place because selecting *prove* or *diagnose* as the semantic of a contract assertion's only evaluation forces a choice we do not want to make. An assertion we want diagnosed when it might be violated is very often one we also want enforced at runtime when it is, and an assertion we want proven at each call gains nothing from an attempt to prove it in the callee.

Each contract assertion therefore has an additional evaluation, called the analysis evaluation, around which the configuration of Section 2.3 and the label facet of Section 2.2.1 are built:

- The analysis evaluation occurs with a purely compile-time semantic: *ignore*, *diagnose*, or *prove*, and is designed to be the first notional evaluation of any given contract assertion.
- For a precondition it takes place at each call, in the caller, once the arguments have been initialized. For a postcondition or a `contract_assert` it takes place in the function body, where a callee-side check would be.
- It always happens, with the semantic selected for it or a stronger one, where *diagnose* is stronger than *ignore* and *prove* is stronger than both. Its semantic can be selected by the configuration (Section 2.3) and by the labels of the assertion (Section 2.2.1), and the strongest of those selections applies, adjusted to the semantics that the labels allow (Section 2.3.1).
- *Prove* and *diagnose* are not typically applied to any other evaluation of the assertion. A configuration or label that would give a runtime evaluation one of them instead contributes it to the analysis evaluation, and that runtime evaluation becomes *ignore* (see Section 2.3.1).

We always want the analysis evaluation to happen as early as possible, since that is when there is the most information with which to prove the assertion. A caller knows the arguments it passes and what it has established about them, and a function body knows what it computes. Once a call crosses into a callee — which might be compiled in another translation unit, with another configuration — the context of the caller is lost, and with it most of what a proof of a precondition could rely on. Placing the analysis evaluation of a precondition in each caller, and that of a postcondition or a `contract_assert` in the function body, makes it at the earliest point at which each kind of assertion can be identified.

Placing the analysis evaluation of a precondition at each call relies on the call identifying the function it calls. When the address of a function is taken and the function is later called through the resulting pointer, that call is anonymous: it cannot see the function's preconditions, so it cannot make their analysis evaluation. The first points at which those preconditions can be identified for

such a call are where the address was taken and the entry to the function itself, and neither knows anything about the arguments the call will pass. This erasure leaves no context at all for a proof, so for calls through the pointer a precondition can be proven only if it is *tautological*.

By *tautological* we mean a predicate that the compiler can, and *does*, prove always true, whatever the values of the parameters and of anything else it reads. Whether to attempt that is the compiler's choice: a compiler may validly decline to try, and always say that it cannot prove such a precondition, so a predicate such as `x == x` is tautological only to a compiler that bothers to show that it is.

It is not the act of taking the address of a function that is a problem, and doing so is not ill-formed. What follows from it is that the function's preconditions must then be proven with no context, so a precondition that is to be proven, and is not tautological, cannot be proven for calls through the pointer. How hard an implementation tries to prove such preconditions, and when it reports one that it cannot, is up to the implementation; Section 4.2 describes what ours do.

The destructor of an object of static or thread storage duration is erased in the same way. It is the one function the program itself hands to someone else to call: it is invoked at exit, or when the thread ends, by the implementation, through the destructor's address, with no caller in the program at which an analysis evaluation could take place. The preconditions of such a destructor must therefore be proven with no context as well — for a declared object, and equally for a temporary bound to a reference of static or thread storage duration, since a lifetime-extended temporary is destroyed in exactly the same way as a declared object with that storage duration.

Virtual functions, with support for contract assertions on them as proposed in [P3097R3] (approved for C++29), erase the function being called in a similar way. The virtual table of a class effectively takes the address of every overrider in it, and a call through a reference or pointer to a base class `B` reaches the final overrider `D::f` without knowing which function that is, while evaluating the contract assertions of both `B::f` and `D::f`. Treated like any other address, that would require every precondition of every overrider to be tautological. We can do better, because a virtual call is not anonymous: it names `B::f`, whose contract the caller can see, so the analysis evaluation of `B::f`'s preconditions takes place at each such call, just like that of any other precondition. What no caller can analyze is whether `D::f` honors the contract of `B::f`, and that can instead be analyzed where `D::f` is defined: its body is analyzed once more, in an *override shadow* that takes the preconditions of `B::f` as given, and proves from them that the preconditions of `D::f` hold and that `D::f` establishes the postconditions of both functions. Both of our implementations use override shadows. Section 1.3 works through an example, and Section 3.2 describes these obligations in detail. A pointer to a member function, however, is an address like any other, even when the member function is virtual.

During constant evaluation there is no analysis evaluation, and nothing to prove an assertion against other than the one evaluation at hand. When the semantic selected for the evaluation of a contract assertion during constant evaluation is nevertheless *prove* or *diagnose* — from a configuration entry that also matches runtime evaluations, or through the labels of the assertion — the predicate is evaluated for that one exemplar:

- *Prove* is treated as *enforce*: a proof is required, so this evaluation must not violate the assertion, and a violation makes the program ill-formed, just as it would with *enforce*.
- *Diagnose* is treated as *observe*: a counterexample that violates the assertion is diagnosed with a warning, and constant evaluation continues, just as it would with *observe*.

Back at runtime, when the analysis evaluation proves an assertion soundly (in the sense described in Section 4.5), no runtime evaluation that the proof covers can result in a violation, and the implementation may drop it — but only within the code the proof was made in. For a postcondition or a `contract_assert`, that is the callee-side check in the same function body, and for a precondition, a caller-side check at the same call. A caller's proof never removes the callee-side check of a precondition: the callee might be compiled in another translation unit, with another configuration, and other callers might not have proven anything. Where an implementation also turns proofs into assumptions (Section 4.6), the dropped checks follow naturally. A conditional proof drops nothing.

The only way a callee-side check is skipped because of the analysis evaluation is through the labels of the assertion, which every translation unit sees in the same way. A label that allows no runtime semantic, such as `always_prove`, guarantees that every translation unit gives the assertion an analysis evaluation with *prove* or *diagnose*, and so its runtime evaluations are always *ignore* (see Section 2.3.1). Configuration alone never has that effect.

2.2.1 Controlling the Analysis Semantic with Labels

A label can already influence the analysis evaluation through its allowed set, as `always_prove` does, but doing so takes over the runtime evaluations as well. To ask that an assertion be diagnosed (or proven) while leaving its runtime evaluations to the build configuration, we propose a new facet, on top of those of [P3400R5], which an assertion-control object opts into by modelling the following concept:

```
namespace std::contracts::labels {
template <typename T>
concept analysis_semantic_label =
  assertion_control_object<T> &&
  requires(const T t) {
    { t.analysis_semantic() } -> std::convertible_to<evaluation_semantic>;
  };
}
```

- If an assertion-control object does not model this concept, the analysis evaluation has its configured semantic, adjusted to its allowed set.
- If it does, `analysis_semantic()` must be a constant expression whose value is *ignore*, *diagnose*, or *prove*, or the program is ill-formed. The analysis evaluation takes the stronger of that value and its configured semantic (step 3 in Section 2.3.1), which is then adjusted to its allowed set as usual.
- Unlike `compute_semantic`, `analysis_semantic` is not passed the configured semantic, and it can only strengthen the analysis evaluation, never weaken it — a label can ask for at least *diagnose* without discarding a configured *prove*. A label that needs to suppress analysis can do so through its allowed set.
- A combined assertion-control object models this concept if either of its constituents does, and its `analysis_semantic` returns the stronger of theirs: *prove* combined with anything is *prove*, and *diagnose* combined with *diagnose* or *ignore* is *diagnose*.

A label asking for an assertion to be diagnosed, while it is checked (or not) at runtime as the build configures, is then:

```
namespace std::contracts::labels {
struct diagnosed_t {
  using assertion_control_object = diagnosed_t;

  constexpr evaluation_semantic analysis_semantic() const {
    return evaluation_semantic::diagnose;
  }
};
constexpr diagnosed_t diagnosed{};
}

void f(int x)
{
  contract_assert<diagnosed>(x > 0);  // OK, warns: cannot be proven
}
```

Built to enforce contract assertions, this warns that the assertion cannot be proven, and `f(-1)` still results in a contract violation at runtime: https://godbolt.org/z/s47Mfqef3. The Standard Library could provide `diagnosed`, along with a corresponding `proven` label returning *prove*.

Proposal (on top of P3400): Analysis-semantic control

Allow an assertion-control object to strengthen the semantic of the analysis evaluation of its contract assertion with a `constexpr analysis_semantic` member function. Combined objects use the stronger of their constituents' analysis semantics.

2.3 Analysis Semantic Configuration

There are several ways a [P3595R1] configuration could select the semantic of the analysis evaluation. The first we considered was to let the `semantic` of an output be a comma-separated list, such as `"diagnose,enforce"`, giving a compile-time semantic followed by a runtime one. On examination, that is the only combination that makes sense — `"prove"` needs nothing after it, and `"diagnose,ignore"` is just `"diagnose"` — and the list ties both choices to a single match, so turning on diagnosis for every assertion means rewriting every entry that selects a runtime semantic. A second option, an `analysis` key in an entry's output alongside its `semantic`, has the same coupling. We instead propose that the analysis evaluation be selected by a lookup of its own through the same list of entries, distinguished by a new match key, so that diagnosis can be turned on for every assertion with a single entry while every runtime configuration is left untouched:

```
...
    "match": {
        ...
        "analysis" : "boolean"
    },
...
```

- Like `caller`, `analysis` is opt-in. The lookup for an analysis evaluation considers only the entries whose `match` has `"analysis": true`, and the lookup for any other evaluation skips them (`"analysis": false` is the same as omitting the key). When no entry matches, the analysis evaluation has the *ignore* semantic.
- `analysis` and `caller` cannot both appear in the same `match`, and an entry with both is a configuration error. The position of an analysis evaluation is fixed by the kind of the assertion, so `caller` could only restate or contradict it; to analyze only preconditions, match on `"kind": "pre"`.
- An analysis evaluation never happens during constant evaluation, so an analysis entry never matches if it also has `"constexpr": true`. Any other entry with `"constexpr": true` that selects *prove* or *diagnose* is a configuration error, since it asks for something that cannot happen. An entry without a `constexpr` key that selects either semantic, such as the one `-fcontract-evaluation-semantic=prove` stands for, still applies to constant evaluation as described in Section 2.2.
- The output of an entry with `"analysis": true` must have a `semantic` of `ignore`, `diagnose`, or `prove`, and no `dynamic`; anything else is a configuration error. An analysis evaluation happens entirely at compile time, so there is nothing for a dynamic selection to select.

As with `-fcontract-evaluation-semantic`, a flag can stand for an entry appended after those of the configuration files:

```
-fcontract-analysis-semantic=<semantic>
```

which is equivalent to:

```
{"match": {"analysis": true}, "output": {"semantic": "<semantic>"}}
```

With this, diagnosing every contract assertion that cannot be proven, while still enforcing every one of them at runtime, takes one entry in addition to the runtime configuration:

```
[
  {"match": {"analysis": true}, "output": {"semantic": "diagnose"}},
  {"output": {"semantic": "enforce"}}
]
```

Every contract assertion is then diagnosed where it cannot be proven — each precondition at each call — and the program still enforces each of them when it runs: https://godbolt.org/z/jj753ns3r.

The recommendation of the *Evaluation point* discussion in Section 3.1, proving preconditions at call sites and postconditions in function bodies, needs no `caller` at all, since that is where those analysis evaluations already are; here every other assertion is diagnosed, and the runtime evaluations are left to the rest of the build configuration:

```
[
  {"match": {"analysis": true, "kind": "pre"},
   "output": {"semantic": "prove"}},
  {"match": {"analysis": true, "kind": "post"},
   "output": {"semantic": "prove"}},
```

```
  {"match": {"analysis": true}, "output": {"semantic": "diagnose"}}
]
```

Since the entries are matched in order, a library whose assertions a user does not want analyzed — often the case for third-party headers, whose internal code paths the user cannot change — is excluded with an earlier entry:

```
[
  {"match": {"analysis": true, "namespace": "thirdparty"},
   "output": {"semantic": "ignore"}},
  {"match": {"analysis": true}, "output": {"semantic": "diagnose"}}
]
```

2.3.1 Adjusting to the Allowed Semantics

[P3595R1] adjusts a semantic that is not allowed for a contract assertion to one that is. With the analysis evaluation separated from the runtime evaluations, a single allowed set no longer serves both, and treating them together gives results we do not want. A label allowing *enforce* and *prove* should not let a build that configures *ignore* turn every unprovable assertion into an error when *enforce* is available, and a label allowing only *prove* should not be an error simply because it allows no runtime semantic.

We therefore split the allowed set A of an assertion (every semantic, when no label restricts it) into its runtime part R, the semantics other than *diagnose* and *prove*, and its analysis part N, those two. *Ignore* — not evaluating the assertion at all — is added to a set only when we know that another evaluation of the assertion will have a semantic that covers it:

- The analysis evaluation may be *ignore* if R is not empty, since a runtime evaluation will then have an allowed semantic.
- A callee-side runtime evaluation may be *ignore* if R is empty, since the analysis evaluation must then have the *diagnose* or *prove* semantic in every translation unit. Whether a callee-side evaluation may be skipped is decided only by the labels of the assertion, never by the configuration of the analysis evaluation, since the analysis evaluation of a precondition happens in its callers, which might be compiled with other configurations.
- A caller-side runtime evaluation may always be *ignore*, since the callee-side evaluation will be made with an allowed semantic.

If both R and N are empty, the program is ill-formed.

The semantics of the evaluations of an assertion are then determined as follows, where *diagnose* is stronger than *ignore* and *prove* is stronger than both:

1. The lookup for the analysis evaluation gives its configured semantic (*ignore* if no entry matches), and the lookup for a runtime evaluation gives that evaluation's configured semantic.
2. If the configured runtime semantic is *prove* or *diagnose* — from a configuration written without the `analysis` key, for example, or from `-fcontract-evaluation-semantic=prove` — the analysis evaluation takes the stronger of it and its own configured semantic, and the configured runtime semantic becomes *ignore*.

3. If the assertion-control object provides an analysis semantic (Section 2.2.1), the analysis evaluation takes the stronger of that and its configured semantic.

4. The analysis semantic is adjusted to the allowed set of the analysis evaluation, N (plus *ignore* when R is not empty). *Prove* falls back to *diagnose* and then *ignore*; *diagnose* falls back to *ignore* and then *prove*; *ignore* falls back to *diagnose* and then *prove*.

5. The configured runtime semantic is adjusted to the allowed set of the runtime evaluation as described in [P3595R1], considering only the runtime semantics, so a runtime evaluation never falls back to *prove* or *diagnose*.

6. The adjusted runtime semantic is passed to the assertion-control object's `compute_semantic`, if it has one. A result of *prove* or *diagnose* is moved to the analysis evaluation as in step 2, and the stronger semantic that results must be in the analysis evaluation's allowed set, and the runtime evaluation then becomes *ignore*. That *ignore*, like any other result, must be in the runtime evaluation's allowed set — the input to `compute_semantic` comes from the configuration, so its result cannot be what lets a callee-side check be skipped. Otherwise, the program is ill-formed.

A few cases show how this works out:

- An assertion with no label has every runtime semantic allowed, so its runtime evaluations are configured exactly as they are today, and its analysis evaluation has whichever of *ignore*, *diagnose*, or *prove* is configured for it.
- With `always_prove`, R is empty, so the analysis evaluation must be *prove*, and the runtime evaluations can only be *ignore*.
- With `terminating`, extended to allow *prove* as we propose in Section 2.4, a configured analysis semantic of *prove* is kept, and a configured analysis semantic of *diagnose* falls back to *ignore*. Either way its runtime evaluations are adjusted to *enforce* or *quick-enforce*, just as they are today.
- With `symbolic`, extended to allow *prove* and *diagnose*, the analysis evaluation has whichever of the three is configured, and the runtime evaluations are *ignore*, just as they are today.
- A label whose allowed set contains neither *prove* nor *diagnose*, such as `always_enforce`, has an empty N, so its assertions are never analyzed.

Proposal (addition to P3595): Configure the analysis evaluation

Extend the configuration of [P3595R1] with an `analysis` match key, selecting the semantic of the analysis evaluation with a separate, opt-in lookup through the same entries, and with a `-fcontract-analysis-semantic` flag. Adjust the semantics of the analysis and runtime evaluations to separate allowed sets, as described in this section.

2.4 Other Interactions

The new semantics also affect the labels of [P3400R5], which are specified in terms of evaluation semantics. The `hardened` label, which provides only a `compute_semantic` and no allowed set, is

unaffected.

Under the rules of Section 2.3.1, a label that restricts the semantics of an assertion must allow *prove* or *diagnose* for that assertion to be analyzed at all. Two of the labels of [P3400R5] should do so.

The `terminating` label allows only *enforce* and *quick-enforce*, guaranteeing that execution never continues past a violation of the assertion. An assertion that has been proven can never be violated, which is at least as strong a guarantee, so we propose adding *prove* to the allowed set of `terminating`. This changes nothing for a build that does not ask for proofs: the analysis evaluation of a terminating assertion falls back to *ignore*, and its runtime evaluations are adjusted to *enforce* or *quick-enforce* just as they are today. When the analysis evaluation of a terminating assertion is configured with *prove*, the assertion must be proven — each precondition at each call — and it is still enforced at runtime, since whether a callee-side check is skipped is never decided by configuration: https://godbolt.org/z/KhKnbT547.

We do not add *diagnose* to the allowed set of `terminating`. A terminating assertion evaluated with *diagnose* would produce a warning and then be ignored at runtime, which is exactly what the label exists to prevent. Allowing *diagnose* together with *enforce* — diagnosed at compile time and still enforced at runtime — would be useful, but expressing that a label requires both needs a deeper change to the allowed-semantics model, which we leave for future work.

The `symbolic` label allows only *ignore*, so that the predicate of the assertion is never evaluated, and may name functions that are declared but never defined. Neither *prove* nor *diagnose* evaluates the predicate either, and assertions that are never checked at runtime are exactly those for which a compile-time proof, or a diagnostic when there is none, is most valuable. We therefore propose adding both *prove* and *diagnose* to the allowed set of `symbolic`. Its runtime evaluations remain *ignore*, and its analysis evaluation has whatever semantic is configured for it. (A predicate that calls a function whose definition is not available can, of course, rarely be proven soundly.)

The `always_` labels of [P3400R5] each force a single semantic, and we leave them as they are; their assertions are never analyzed. The `never_` labels, and the `evaluation_semantic_set::all()` factory, are specified in terms of every enumerator of `evaluation_semantic`, so they include the new semantics without change (although the illustrative implementation of `evaluation_semantic_set` in [P3400R5] would need room for them).

Proposal (on top of P3400): Allow analyzing terminating and symbolic assertions

Add *prove* to the allowed semantics of the `terminating` label, and add *prove* and *diagnose* to the allowed semantics of the `symbolic` label.

3 Discussion

3.1 Design of the Semantics

- **Naming**: Both names for the semantics were chosen to follow the existing pattern for evaluation semantic names where they are verbs that are applied to the assertion. In the same way we can enforce the truth of the predicate, we can prove its truth or diagnose when it is not true.

Prove is the most natural of the two names, and diagnose is our current best placeholder for the observe-equivalent of prove. Diagnose, however, is likely to be much more important as it seems essential to be able to deploy code portably when proof can often vary with even minor changes in build parameters.

- **Evaluations**: It is key to recognize that the evaluation of a *proven* contract assertion prevents the need to consider what else happens. All evaluation semantics share the property that when the contract assertion is not violated they can do nothing (even the checked semantics can elide any predicate if they know it will not be violated beforehand). Therefore when we select to have a contract assertion *proven* we do not need to specify anything else.

  When we only ask to *diagnose*, however, we are left with a lingering question as to what to do with the assertion at runtime after a diagnostic was emitted at compile time. That question is, of course, based on different criteria and users can easily want to do any of the runtime semantics even after having diagnosed the potential failure at compile time. The C++26 Contracts model provides an easy mechanism to enable this, however, by simply evaluating the contract assertion again with one of the runtime semantics (or not, which would be equivalent to any number of *ignored* evaluations). That is exactly what the analysis evaluation of Section 2.2 does: *diagnose* is applied in the first evaluation of the assertion, and its runtime evaluations follow with whatever semantics are configured for them.

- **Evaluation point**: Proving a precondition callee-side is often impossible for non-tautological predicates. Proving a postcondition caller-side is similarly often impossible. From a standardization perspective, we do not recommend avoiding this possibility — but instead suggest that when building configuration options for selecting the *prove* semantic we do so in ways that primarily *prove* preconditions at call sites and postconditions in the function body. The analysis evaluation of Section 2.2 builds this recommendation in, since its position is fixed at exactly those points.

3.2 Virtual Functions

The override shadows of Section 2.2 give each overrider `D::f` three kinds of obligation. For each function `B::f` that `D::f` overrides, directly or indirectly, the override shadow takes the preconditions of `B::f` as given and then, in order, attempts to prove:

1. The preconditions of `D::f`, at entry — a call through `B` that satisfies `B::f`'s preconditions must not violate those of `D::f`.
2. The postconditions of `D::f`, at each return — these are evaluated for a call through `B` too, where only `B::f`'s preconditions are known to hold.
3. The postconditions of `B::f`, at each return — the caller relies on them, and they are evaluated after `D::f` returns.

Each of these is an analysis evaluation of the assertion being proven, with the semantic resolved for it as usual (Section 2.3.1), so a postcondition of `B::f` with the `always_prove` label must be proven in every override, while one whose analysis evaluation is *ignore* is not analyzed at all. When every such evaluation is *ignore*, there is no override shadow. A `B::f` with no contract assertions of its own still gets one: nothing is given, so the preconditions of `D::f` must then be proven tautological — a

call through `B` may pass any argument at all. As in any other shadow, an assertion proven earlier is a premise for the later ones, and whether a proof that relies on the preconditions of `B::f` is sound or only conditional follows the rules of Section 4.5. The diagnostics are issued where `D::f` is defined, since that is where each override shadow is analyzed.

Every function that `D::f` overrides gets its own override shadow. If `D` derives from `C`, which derives from `B`, a call through `B` evaluates the contract assertions of `B::f` and `D::f` but not those of `C::f`, so `D::f` is analyzed against `B::f` and against `C::f` separately, and the override shadows of `C::f` against `B::f` do not cover `D::f`. The same holds for each base of a class with multiple bases, as in Section 1.3. When the contract assertions of `B::f` name `this`, a `B` that occurs as several subobjects of `D` gets an override shadow for each, since each reads a different subobject.

Within an override shadow, `this` refers to the `B` base of the object, and when `D::f` has a covariant return type, the postconditions of `B::f` see the result converted to the return type of `B::f`. A pure virtual `B::f` has no body to analyze for its own sake, but its contract assertions are still the given preconditions and the obligations of every override.

An override shadow is one more optimized copy of `D::f` for every function it overrides, so it is a proving strategy with a real cost, and an implementation should let it be turned off on its own. Without it, the same obligations remain but are decided more cheaply. The preconditions of `D::f`, for a call through `B`, can then be proven only when they are tautological; any other is diagnosed as unproven. The postconditions of `B::f` become additional obligations of the ordinary analysis of `D::f`'s body, with nothing given — `D::f` must establish them even for arguments that do not satisfy the preconditions of `B::f`. This rejects some overrides that are correct for every call through `B`, but it costs no more than proving `D::f`'s own postconditions, and it does not reject an override merely because `B::f` has a postcondition.

3.3 Premises and Assumptions

- **Previous Assertions**: An open question is how we would want to treat earlier contract assertions that were ignored or observed, and whether the truth of their predicates should be considered as part of the proof of the contract assertion. A compiler could choose to not consider such predicates, in which case it will be able to prove fewer assertions when mixing evaluations with the *prove* semantic with those that are *ignored* or *observed*. It is, however, much more likely to avoid undesired false positive diagnostics if implementations are given freedom to consider any contract assertion as not having been violated when considering a proof of a later contract assertion. It is up to the user to decide on a path to gain confidence in the earlier contract assertion, and then build confidence in later ones contingent on the earlier ones.
- **Assumption**: Once a contract assertion is proven, it can also be assumed for the sake of later static analysis — eliding branches that only get taken if the assertion could be false, for example. The *assume* semantic will help greatly for full-program optimization if users are able to combine caller-side precondition checking with callee-side precondition assumption, for example, as long as such users avoid attempting to mix build configurations. (Of course, even when mixing build configurations the bad outcomes will only occur when there is a contract violation — code with no violations will still behave correctly).

Of course, there is no *assume* semantic allowed for standard explicit contract assertions, and so this is a use of the feature that must be provided by extensions or alongside future proposals.

3.4 Portability

One major concern with leaving it up to QoI to identify the cases where an assertion is provable or not is that the feature itself becomes hard to use at scale and in practice. Having code that compiles at one optimization level and not another — or worse compiles successfully in one run then fails if rebuilt with the same settings — is challenging. Using the optimizer to generate proofs can lead to all of these downsides — some optimizations are heuristic, or stop trying based on available time or memory budgets, and can be highly chaotic regarding when they do or don't arrive at certain conclusions.

On the other hand, this is a common problem with all of the papers that are proposing alternatives that take a similar approach. Given that, if there is a desire to provide a Standardized approach to this kind of functionality, this paper is a viable way to do it. If the lack of repeatability and portability is a concern, compilers are still free to provide the semantics proposed here as their own extensions, giving people the benefit without the associated perceived promises that making the facility part of the Standard might convey.

4 Implementation Experience

This proposal has been implemented in the P3850 branches of GCC and Clang that are available on Compiler Explorer. Both make the two semantics, the analysis evaluation, and its configuration available with the `-fcontracts-p4405` command-line flag, and also with the flag `-fcontracts-p3850` that enables prototype implementations of the other papers described in the overall plan in [P3850R1]. Both predefine the feature-test macro `__cpp_contracts_prove` when the semantics are enabled (with the implementation date as its value). Both implement all of the proving strategies described below, including the override shadows of Section 2.2, though not identically, and the examples show the two side by side; where they differ, we say so.

A fork of the EDG front end implements everything that does not need a proof: the `analysis` key and `-fcontract-analysis-semantic`, the split allowed sets and the resolution of Section 2.3.1, the `analysis_semantic` facet, the rules for constant evaluation, the diagnostics for taking the address of a function (Section 4.2), and the obligations of every override. It implements no proving strategy, so with its C-generating back end — the one available on Compiler Explorer — every analysis evaluation with *prove* is an error and every one with *diagnose* is a warning. Paired with our GCC as its back end, the EDG front end leaves the analysis evaluations to GCC, which makes them as it would for its own front end.

4.1 The Semantics and the Analysis Evaluation

The two new semantics were added as ordinary `evaluation_semantic` enumerators,[1] which let them flow through the entire existing resolution pipeline — the `-fcontract-evaluation-semantic=` default,

[1] As with every experimental semantic, the prototype's enumerator values follow those of the other experimental semantics the branches carry, so they differ from the values in our proposed wording.

per-group semantics, JSON configuration files as proposed in [P3595R1], and the assertion-control labels of [P3400R5] (both the `allowed_semantics` and `compute_semantic` facets). Each contract assertion then carries two semantics: that of its analysis evaluation, which is always *ignore*, *diagnose*, or *prove*, and that of its runtime evaluations, which is never *diagnose* or *prove*. Both compilers resolve them exactly as Section 2.3.1 describes, with the analysis lookup, the move of a configured runtime *prove* or *diagnose* into the analysis evaluation, the `analysis_semantic` facet of Section 2.2.1, and the split allowed sets, so a label such as `always_prove` works whatever semantic the build configures. The library's `terminating` and `symbolic` labels allow the semantics proposed in Section 2.4.

The analysis evaluation never evaluates the predicate. It is placed as Section 2.2 describes, and it is always the first evaluation, so a proof never relies on the assertion's own runtime check:

- The analysis evaluation of a precondition is made at each call, in the caller, with the arguments known: at a direct call, a member or operator call, a virtual call (for the function the call names), a constructor call, and a destructor call, whether written or implicit, as at the end of an automatic object's lifetime or in a `delete` expression. It is never made in the callee.
- The analysis evaluation of a postcondition or a `contract_assert` is made in the function body, ahead of its runtime evaluation.
- The runtime evaluations are emitted as they would be without the analysis evaluation, each with its own semantic.

The same function, with a precondition that cannot be proven and a postcondition that can, shows the two semantics and their independence from the runtime evaluations. With *prove* selected for every analysis evaluation the program is ill-formed at the call of `f`, and with *diagnose* it builds with a warning there; with the runtime evaluations *ignored* it then runs without ever calling `check`, and with them *enforced* it reports a violation of the precondition when it runs: https://godbolt.org/#z:OYLghAFBq....

```
bool check(int x);  // Has side effects

int f(const int x)
  pre(check(x))        // Cannot be proven
  post(r: r % 2 == 0)  // Proven
{
  return x * 2;
}
```

There is no analysis evaluation at an indirect call, in a P3097 interface wrapper, or at a call in a manifestly constant-evaluated context (the initializer of a `constexpr` or `constinit` variable, a `static_assert`, a template argument, or an immediate function, for example). The diagnostics for taking the address of a function (Section 4.2) cover the unary `&`, the conversion of a function to a pointer, forming a pointer to a member function, binding a reference to a function — so passing a function to `std::function`, `std::invoke`, `std::thread`, or `std::bind` counts — and converting a lambda to a pointer to function. A closure handed to such a facility is not a function, and the library calls its function call operator directly, so that call has an analysis evaluation of its own, inside the library: an `always_prove` precondition on a lambda that is not a tautology is an error there, reported with the instantiation context that leads back to the user's code.

Both compilers apply the same diagnostics (Section 4.2) to the objects of static and thread storage duration whose destructors they register to be called at exit or at the end of the thread, including lifetime-extended temporaries. For an array, both register a generated function that calls the destructor of each element directly, and those calls are analyzed as well, so such a definition can draw a second diagnostic. Which objects are registered differs between the compilers in one case, described in Section 4.2. A destructor call that is never made is not analyzed: neither that of a temporary whose copy is elided nor a constant destruction, during which there is no analysis evaluation (Section 2.2), draws a diagnostic.

The compilers also differ in which functions they analyze. GCC and the EDG front end process the body of every inline function they see, so they diagnose the analysis evaluations of the postconditions and assertions in an inline function that is never used; Clang generates code only for the functions it emits, so it does not. Since the body of an inline function is code the program could execute, we consider either behavior reasonable and have left the difference in place. Either way a template is analyzed per instantiation, an uninstantiated template not at all, and nothing is analyzed under `-fsyntax-only` (although the diagnostics for taking the address of a function, which need only folding, are still issued).

During constant evaluation all three implementations follow Section 2.2: a configuration entry with `"constexpr": true` that selects *prove* or *diagnose* is a configuration error, and when constant evaluation otherwise reaches *prove* or *diagnose* — here from `-fcontract-evaluation-semantic=` — it is evaluated as *enforce* or *observe*, respectively: `https://godbolt.org/#z:OYLghAFBq...`. A dynamic selection, made at link or run time, never selects the semantic of an analysis evaluation, and a selection function that returns *prove* or *diagnose* is treated as returning *ignore*.

In C, for which both compilers implement contract assertions as well, the analysis evaluation of a precondition stays in the function body, ahead of its runtime evaluation, and taking the address of a function is never diagnosed. GCC's C front end keeps a function's contract assertions only on its definition, so a call cannot see the preconditions of the function it calls, and the analysis evaluation at each call that Section 2.2 recommends is not possible.

4.2 Taking the Address of a Function

Our implementations diagnose the preconditions of a function whose address has been taken only where it is *known* that the analysis evaluation of a precondition for calls through the pointer uses *prove* or *diagnose*, and that is the case only when the labels of the precondition force it: when they allow no runtime semantic, as `always_prove` and `always_diagnose` do, every translation unit gives the precondition an analysis evaluation with *prove* or *diagnose*, and nothing ever checks it at runtime. Such a precondition must then be proven tautological. Where the address of the function is taken, our implementations report that the precondition cannot be proven for calls through the pointer — an error with *prove*, a warning with *diagnose* — unless folding proves the precondition always true, in which case nothing is reported. Folding is the only strategy that can help here, since there is no call to give the proof shadow any context, so when GCC or Clang is given `-fno-contracts-prove-fold`, even `pre<always_prove>(true)` is diagnosed. The diagnostic is about the precondition, not about taking the address, which remains perfectly valid: it reads "contract assertion cannot be proven for a call through the address of `f`", or "contract predicate is always false for a call through the address of `f`" when the predicate folds to `false`, followed by a note pointing at the precondition. When

one function has several such preconditions, only the first that is not proven is reported, preferring one with *prove* to one with *diagnose*. The warning is controlled by `-Wcontract-unproven`, like every other unproven analysis evaluation with *diagnose*; the separate `-Wcontract-unanalyzed` warning that earlier versions of our implementations issued for configuration-selected preconditions no longer exists. Since folding is done by the front end, these diagnostics are issued under `-fsyntax-only` as well.

When only the configuration selects *prove* or *diagnose* for the analysis evaluation of a precondition, nothing is reported where the address is taken, and calls through the pointer simply go unanalyzed. The configuration of the translation unit that takes the address does not determine what calls through the pointer will be: the precondition is evaluated in the callee, with whatever semantic is selected where the body of the function is compiled, and that is the one place where it could be known how such calls are treated. For a function that is not inline, that place is removed from the taking of the address, often in another translation unit entirely.

The same holds for the destructors of objects of static and thread storage duration, including lifetime-extended temporaries: the definition of such an object, or the binding of such a temporary, is diagnosed only when a precondition of the destructor allows no runtime semantic and cannot be folded to `true`. This is the same line that our implementations draw for virtual functions, where a `B::f` with no contract assertions gives nothing to prove from, and where override shadows are turned off: in both cases the preconditions of the overrider must be tautological.

```
void f1(int x) pre<always_prove>(sizeof(x) == sizeof(int));
void f2(int x) pre<always_prove>(x > 0);
void f3(int x) pre<always_diagnose>(x > 0);
void f4(int x) pre(x > 0);  // Analyzed with prove by the configuration

int n;

struct S {
  ~S() pre<always_prove>(n > 0);
};

S s;  // Error, cannot be proven for the call at exit

void g()
{
  void (*p1)(int) = &f1;  // OK, proven tautological
  void (*p2)(int) = &f2;  // Error, cannot be proven through p2
  void (*p3)(int) = &f3;  // OK, warns: cannot be proven through p3
  void (*p4)(int) = &f4;  // OK, calls through p4 are not analyzed
}
```

Built with `-fcontract-analysis-semantic=prove`, which is what selects *prove* for the precondition of `f4`, both compilers accept the address of `f1`, whose precondition folds to `true`, and of `f4`, whose precondition is analyzed with *prove* only because the configuration says so; they reject the address of `f2` and the definition of `s`, and warn about the address of `f3`: https://godbolt.org/#z:OYLghAFBq....

Each implementation diagnoses exactly the objects whose destructors it actually registers to be

called through their addresses, and the diagnostic is followed by a note saying so: "`s`, of static storage duration, is destroyed by a call through the address of its destructor". They therefore differ on an object that is not `constexpr` but whose destruction is a constant expression. Clang destroys such an object by constant evaluation and never at exit, so it does not diagnose it, while GCC and the EDG front end evaluate constant destruction only for a `constexpr` variable, register the destructor of any other object, and diagnose it:

```
int g;

struct DE {
  constexpr ~DE()
    pre<always_prove>(__builtin_is_constant_evaluated() || g > 0) {}
};

constexpr DE ce1;  // OK, destroyed by constant evaluation
DE ce2;            // Error in GCC and EDG, OK in Clang
```

Either way, the diagnostic is about a call that is actually made through the destructor's address, so we have left the difference in place.

4.3 Configuration

The configurations of Section 2.3 work as written in both compilers. The one that follows the *Evaluation point* recommendation of Section 3.1, proving preconditions at call sites and postconditions in function bodies and diagnosing every other assertion, needs no `caller` key, since that is where those analysis evaluations already are. With it, both compilers prove a postcondition from the body of the function that has it, and a precondition at each call where the caller has established it — in the example, through the body of a clamping function inlined into the caller — and report only the calls where it has not: https://godbolt.org/#z:OYLghAFBq.... Clang does not otherwise implement caller-side checking, but it does make the analysis evaluation of a precondition at each call, since that needs no check.

A configuration written without the `analysis` key still works: a configured runtime *prove* or *diagnose* — from `"semantic": "prove"`, or from `-fcontract-evaluation-semantic=prove` — is moved to the analysis evaluation, and when it comes from an entry with `"caller": true`, it is moved to the analysis evaluation at that call.

The examples of [P4363R0], which encourages a diagnostic when a precondition cannot be established, work the same way with *diagnose* selected for the analysis evaluations of preconditions: `func(x)` called from `g(int x)` is diagnosed, the same call guarded by `if (x > 0)` is proven, and `set_data_idx(c, -1, 500)` is diagnosed as always false: https://godbolt.org/#z:OYLghAFBq....

4.4 Proving Strategies

An analysis evaluation with *prove* or *diagnose* is diagnosed unless an enabled proving strategy proves its predicate. In both compilers each strategy is separate and individually controllable, with its own flag that is implied by `-fcontracts-p4405` and has a `-fno-` form:

- `-fcontracts-prove-fold` enables folding;

- `-fcontracts-prove-shadow` enables the proof shadow;
- `-fcontracts-prove-assume` turns sound proofs into assumptions; and
- `-fcontracts-prove-override` enables the override shadows of Section 2.2.

With every strategy turned off, every analysis evaluation with *prove* or *diagnose* is diagnosed, which is still useful for checking that a configuration selects the evaluations it was meant to.

Folding proves any predicate the front end can fold to `true`, whether or not it is a constant expression (`x == x`, `u >= 0u`, `sizeof(T)` in a particular instantiation), and a predicate that folds to `false` is diagnosed as "always false" rather than merely unproven. For the analysis evaluation of a precondition, which is made at the call, each constant argument is substituted for its parameter before folding, which is what diagnoses Example 3 of [P4021R2]:

```
static int divide(int num, int denominator)
  pre(denominator != 0);

int main(int argc, char**)
{
  int num = 10;
  int a = divide(num, 2);                    // OK, proven by folding
  int b = argc > 1 ? divide(num, argc) : 0;  // OK, proven by the shadow
  int c = divide(num, argc);                 // Error, cannot be proven
  int result = divide(num, 0);               // Error, always false
  return a + b + c + result;
}
```

Folding settles the first and last calls. The guarded call is proven by the proof shadow, described next, and with `-fno-contracts-prove-shadow` it is diagnosed too: https://godbolt.org/#z:OYLghAFBq....

GCC folds with its front end's folders. Clang uses the folding mode of its constant evaluator, which cannot reason about a value it does not know, so it folds less: `x == x` and `u >= 0u` are not proven by folding in Clang, though its proof shadow proves them when optimizing.

The *proof shadow* handles what folding cannot. Each function holding an analysis evaluation that folding did not settle is copied, the copy is optimized, and it is never emitted; an evaluation is proven if the optimizers can show that the point where its check would fail is unreachable. GCC makes the copy before its optimizations begin and optimizes it alongside the function, through the same early, interprocedural, and late passes, deciding it during the late ones. Clang moves the copies, with every function they reach, into a separate proof module, which is optimized and decided before the real module is optimized at all, so every proof is known before the code that might use it is compiled.

The shadow is not meant to change the real code: with `-fno-contracts-prove-assume`, that code is what it would be without the analysis evaluations. With `-fcontracts-prove-assume` only a sound proof changes it, in ways described below: the proof becomes an assumption, and in Clang a call to a function another translation unit could provide, whose inlining into the shadow the proof relied on, is inlined in the real code as well.

Because the shadow is optimized like the function, it proves whatever the selected `-O` level can show — nothing at `-O0`, where folding is the only strategy, and, in GCC, less at `-Og` than at `-O1` or `-O2`

(Clang's `-Og` is its `-O1`, so it proves the same there): https://godbolt.org/#g:!((g:!((g.... This is the same dependence on build parameters that the approaches of [P4021R2] and [P4019R1] have, and it is a large part of why we expect *diagnose* to be the more widely used of the two semantics. All diagnosis happens when each translation unit is compiled, even under link-time optimization. GCC then decides its shadows after only the early optimizations, so it can prove less with `-flto` than without it (the optimization-level example is proven at `-O1` but not at `-O1 -flto`); Clang's proof module is optimized in full when the translation unit is compiled, so it proves the same either way.

Since a shadow's result is known only once it has been optimized, when shadows are in use GCC issues every *prove* and *diagnose* diagnostic at the end of the compilation, and Clang issues them from its back end. Each still carries the template instantiation context that an immediate diagnostic would — GCC's "required from" lines, and Clang's "in instantiation of" notes.

The analysis evaluation of a precondition is placed in the caller's shadow, before the call, with the arguments substituted for the parameters. That is what makes Example 1 of [P4021R2] — a `static` helper whose assertion relies on its caller's null check — work without needing to inline the helper: the call guarded by the check is proven, and an unguarded call is diagnosed: https://godbolt.org/#z:OYLghAFBq.... The loop from the introduction of [P4021R2] is, as it should be, ill-formed under *prove*, while the same loop with an in-bounds limit is proven: https://godbolt.org/#g:!((g:!((g.... Neither prototype proves the precondition of `lookup(saturate(v))`, though both prove it for `int p = saturate(v);` followed by `lookup(p)` (neither places an argument in the shadow when it cannot be evaluated twice).

Jonathan Grant's own demonstration of `compile_assert` [P4021R2] — a `divide` function and a small bank-account class, whose checks are spelled with macros named `pre`, `post`, and `contract_assert` — carries over directly to contract assertions, with two of its `contract_asserts`, which state what the caller must pass, written as preconditions. With a configuration that proves every precondition at each call and diagnoses every other assertion in its function's body, the calls with valid arguments are proven, and `divide(10, 0)` and `acc.withdraw(1000.0)` are errors (GCC reports both as always false, Clang the second as unproven). `compile_assert` decides each check in the code the optimizer leaves after inlining it into `main`, where the values these particular calls pass are known. Here the remaining assertions are diagnosed as unproven, since each is decided once, in its own function's body, for every possible call: the postcondition of `divide` is false for `divide(10, 3)`, and the assertions about the account's balance hold only given every earlier use of the object — they are class invariants, which no single function body can establish: https://godbolt.org/#z:OYLghAFBq....

Override shadows implement Section 2.2. For each function `B::f` that a virtual function `D::f` defined in the translation unit overrides, GCC makes one more copy of `D::f`, like a proof shadow, and Clang emits one more shadow into its proof module, decided after the proof shadows. In it the preconditions of `B::f` are assumed, and the preconditions of `D::f` at entry, the postconditions of `D::f` at each return, and those of `B::f` after them are obligations, each an analysis evaluation with the semantic resolved for it. Within the override shadow `this` is the `B` subobject, a covariant result is converted as the thunk would convert it, and a by-value parameter that a postcondition of `B::f` names is copied at entry, since that postcondition is checked against the caller's copy of the argument, which `D::f` cannot modify. When the contract assertions of `B::f` name `this`, a `B` occurring as several subobjects of `D` gets an override shadow for each. Virtual destructors, including implicitly defined ones, get

override shadows too, as does each instantiation of an overriding member of a class template. An unproven obligation is diagnosed at the definition of `D::f`, as "contract assertion cannot be proven for a call through `B::f`", with a note at the predicate. An override proof is never assumed, so the real code is unchanged, and an inline `D::f` is analyzed only in a translation unit that compiles it — the one that emits the virtual table of `D`, through which every call through `B` reaches it — not in every translation unit that includes its definition. Both compilers behave as Section 3.2 describes, and with `-fno-contracts-prove-override`, or at `-O0`, both fall back to the cheaper obligations described there, as the example of Section 1.3 shows. The premises of an override shadow are guaranteed, in the sense of the next section, only when the labels of `B::f`'s preconditions settle them, since they are evaluated in each caller's translation unit. Clang's folding decides some obligations as always false that GCC reports as merely unproven.

4.5 Previous Assertions and Undefined Behavior

The shadow assumes the predicates of the contract assertions evaluated before the one being proven, which is one answer to the *Previous Assertions* question of Section 3.3 (it is why the second `contract_assert` in Section 1.1 would be proven even without its `if`). To keep the answer honest, each proof is classified as either *sound* or *conditional*:

- An earlier assertion is *guaranteed* if it was evaluated with *assume*, with a terminating semantic (*enforce*, *quick-enforce*, or *`noexcept`-enforce*), or if its analysis evaluation proved it soundly. An evaluation with *ignore*, *observe*, or *`noexcept`-observe* is not guaranteed, nor is an analysis evaluation that was unproven or only conditionally proven.
- A proof that relies only on guaranteed assertions is *sound*. A proof in a function that assumed any assertion that is not guaranteed is *conditional* — it holds only if those assertions do.
- A predicate that calls a function makes the proof conditional unless the function is known to be free of side effects and to return (so an ordinary accessor such as `std::vector::size` keeps a proof sound).

Both kinds of proof suppress the diagnostic, which is the more permissive answer we argued for above. The distinction matters for what can be done *with* a proof, below. A function's own precondition is guaranteed only if that function evaluates it with a guaranteed semantic, and a callee's postcondition only if the caller evaluates it with one. GCC does not yet feed callee-side preconditions and postconditions evaluated with *ignore* into the shadow at all, so it proves somewhat less than this model allows; Clang does, and proves conditionally from them.

Which earlier evaluations are guaranteed also depends on whose code they are. An inline function, a template specialization, or a weak function may be defined in several translation units, each compiled with its own evaluation semantics, and the linker keeps just one of those definitions. Suppose a caller's proof relied on a check in its own translation unit's copy of such a function, and the linker kept a copy in which that assertion is *ignored*. A program with no undefined behavior in any translation unit would then acquire some through the assumption made from the proof. This is the mixed-mode hazard that the *Assumption* discussion of Section 3.3 warns of for *assume*, and a proof should never be the thing that introduces it. Both prototypes therefore count a premise from an evaluation only when one of the following is true:

- This translation unit decides the evaluation: it is in the function's own body, it is a caller-side

evaluation at a call in this translation unit, or it is in a callee that was inlined into the real code, not merely into the shadow.

- No translation unit can continue past a violation: the assertion's allowed set, which every translation unit sees in its declaration (the labels of [P3400R5]), contains only terminating semantics and *assume*.

An analysis evaluation counts only when it was proven soundly in this compilation, which is never known about another translation unit's evaluations. Language undefined behavior, discussed next, always counts. Nothing derived from such a premise leaves a function another translation unit could provide through a call that was not inlined: a shadow that calls such a function out of line, when that function's body holds evaluations that make premises, proves only conditionally. The first condition covers more than premises: where the shadow inlined such a function, a proof may also rest on what that copy does *not* do, and another translation unit's copy may do more — it may *observe* an assertion this one *ignores*, with a handler that writes to memory — so the proof is only conditional unless the real code inlined it as well. GCC checks that the real code made the same inlining decisions the shadow relied on; Clang, which decides before the real code is optimized, instead makes the real code inline those calls when it is to assume what was proven, and otherwise proves only conditionally.[2]

Undefined behavior is treated just like an earlier assertion that is assumed: a path that has undefined behavior is unreachable in the shadow, so dereferencing `p` proves a later `p != nullptr`, and `x + 1 > x` is proven for a signed `x`. Language undefined behavior counts as a guaranteed premise, so relying on it does not make a proof conditional. Proving that the undefined behavior does not happen is a different and earlier proof — of the corresponding implicit contract assertion of [P3100R9] — not something the later assertion should take on.

4.6 Assuming What Was Proven

The *Assumption* discussion of Section 3.3 observes that a proven assertion can then be assumed. With `-fcontracts-prove-assume`, both compilers do this for *sound* proofs: once the shadow has proven an analysis evaluation soundly, the real function treats the assertion, at that point, as if it had been evaluated with *assume*, as an optimizer assumption whose predicate is never evaluated. Conditional and unproven evaluations, and the proofs of override shadows, assume nothing. This is the "verified `[[assume]]`" use case of [P4019R1] — an assumption that carries no risk of undefined behavior because the compiler has proven it — expressed through the contract configuration rather than a separate facility. It is also how a proof drops a runtime check in the same code (Section 2.2): the assumption comes before the check, which the optimizers then remove. Since a proof that relies on a premise this translation unit does not decide is only conditional, the rule above also keeps an assumption from ever carrying one translation unit's checks into code the linker took from another.

[2] The prototypes had the same hazard outside of this proposal. In GCC, the noreturn violation path of an *enforce* check in such a function let GCC record facts about it — a range for its return value, for example — that callers in the same translation unit used when they called it out of line. In both compilers, an *ignored* assertion had the opposite effect: a body that never calls the violation handler was found not to throw, though another translation unit's copy might *observe* the assertion with a handler that throws. Both prototypes now keep facts from a function that another translation unit could provide, and whose contract evaluations are not settled by their labels, away from callers that do not inline it.

In practice the real function gains only what its optimizers could not already see where the evaluation is, since the proof was found by those same optimizers in the shadow. One case where they cannot is a check made inside a function that the real function calls out of line but the shadow inlines:

```
static int process(int n)
  pre(n > 0)  // Evaluated with enforce
{
  // ...
}

int g(int n) { return process(n + 1); }

int f(int n)
{
  int r = process(n);       // Called out of line
  contract_assert(n > 0);  // Analyzed with prove: proven, sound
  return r + sink(n / 4);
}
```

Since `process` is called from two places and is too large to inline, `f` calls it out of line, and nothing tells `f`'s optimizers that `n > 0` once it returns. Clang optimizes the shadows in their proof module, where `process` has a single caller and is inlined, so the shadow sees the check and proves the assertion soundly — `process` is this translation unit's own code. The proven assertion becomes an assumption and turns `n / 4` into a plain shift; with `-fno-contracts-prove-assume` the division keeps its rounding adjustment: https://godbolt.org/#z:OYLghAFBq....

The example shows nothing in GCC, which splits off the precondition check of `process` and inlines just that into `f`, so its real code sees the check itself and shifts with or without the assumption. GCC optimizes the shadow alongside the function, with the same inliner, so a call the shadow inlines is usually one the real function inlines too; GCC's tests show the gain only by provoking a difference, with an *assume* evaluation that the shadow applies before inlining and the real function only after it. The work on this proposal also gave the *assume* semantic itself a real implementation for contract assertions in GCC, lowered exactly as `[[assume]]` is. Clang does not yet implement *assume* for contract assertions in the code it emits, treating it as *ignore*, so in Clang an *assume* evaluation helps a proof only in its own function's shadow; the example does not rely on it.

4.7 Cost

The shadow compiles each function holding an unsettled *prove* or *diagnose* analysis evaluation a second time, and that is its cost. In a synthetic translation unit of 400 functions, each holding three such evaluations, compile time at `-O2` grew by about 60% with GCC and 51% with Clang; a header-heavy translation unit with only a few such functions showed no measurable difference. Functions with no such evaluation are not copied, and they compiled in essentially the same time with the shadow enabled as without it.

An override shadow is one more such copy of an overrider for each function it overrides. In a synthetic translation unit of 40 classes, each with 10 overriders of virtual functions that have a precondition and a postcondition, and with every analysis evaluation given *diagnose*, the override shadows cost nothing measurable when the overriders have no contract assertions of their own,

since each overrider is analyzed once either way. When each overrider also has a precondition and a postcondition of its own, compile time at `-O2` grew by 31% with GCC and 36% with Clang with override shadows, against the same translation unit without them, about 7 milliseconds for each override shadow in GCC. A header-heavy translation unit, whose library classes have no contract assertions and so get no override shadows, showed no measurable difference.

5 Wording Changes

These wording changes are relative to the C++29 draft sources with git hash 47974a2f, last modified on Wed, 14 Oct 2026 19:31:54 +0200.

Modified Section Contents

6 Basics **[basic] 29**
6.11 Contract assertions [basic.contract] 29
6.11.2 Evaluation [basic.contract.eval] 29

15 Preprocessing directives **[cpp] 30**
15.12 Predefined macro names [cpp.predefined] 30

17 Language support library **[support] 30**
17.10 Contract-violation handling [support.contract] 30
17.10.1 Header `<contracts>` synopsis [contracts.syn] 30
17.10.2 Enumerations [support.contract.enum] 31

Modifications

6 **Basics** **[basic]**

6.11 **Contract assertions** **[basic.contract]**

6.11.2 **Evaluation** **[basic.contract.eval]**

Modify section 6.11.2[basic.contract.eval], paragraph 1:

1 An evaluation of a contract assertion uses one of the following ~~four~~six *evaluation semantics*: *ignore*, *observe*, *enforce*, ~~or~~ *quick-enforce*, *prove*, or *diagnose*. Observe, enforce, and quick-enforce are *checking semantics*; enforce and quick-enforce are *terminating semantics*.

Add paragraphs to and modify section 6.11.2[basic.contract.eval], paragraph 4:

3+a If the evaluation of a contract assertion occurs in a context that is manifestly constant-evaluated (7.7.7[expr.const.defns]) and the evaluation semantic is prove or diagnose, the behavior is instead the same as if the evaluation semantic were enforce or observe, respectively.

4 The evaluation of a contract assertion using the ignore, prove, or diagnose semantic has no effect.

[*Note 2*: The predicate is potentially evaluated (6.3[basic.def.odr]), but not evaluated. — *end note*]

4+a If the predicate of a contract assertion evaluated with the prove or diagnose semantic cannot be proven to always evaluate to `true`, when no earlier contract violations have occurred, a diagnostic is produced. If the evaluation semantic is prove, the program is ill-formed.

4+b [*Note 2+a*: Proving a contract assertion whose predicate is itself not a constant expression depends on analysis of data and control flow that varies across build and optimization levels and can produce inconsistent results. Failure to prove that a contract assertion is inviolable does not mean there are any particular input conditions that would result in a violation. — *end note*]

15 **Preprocessing directives** [cpp]

15.12 **Predefined macro names** [cpp.predefined]

Modify subclause 15.12[cpp.predefined], Table 22 [cpp.predefined.ft]:

Table 1: Table 22 — Feature-test macros [tab:cpp.predefined.ft]

Macro name	Value
⋮ *20 rows elided* ⋮	
`__cpp_constinit`	`201907L`
`__cpp_contracts`	`202606L`
`__cpp_contracts_prove`	`xxxxxxL`
`__cpp_decltype`	`200707L`
`__cpp_decltype_auto`	`201304L`
⋮ *56 rows elided* ⋮	

17 **Language support library** [support]

17.10 **Contract-violation handling** [support.contract]

17.10.1 **Header `<contracts>` synopsis** [contracts.syn]

Modify section 17.10.1[contracts.syn], paragraph 1:

1 The header `<contracts>` defines types for reporting information about contract violations (6.11.2[basic.contract.eval]).

```
// all freestanding
namespace std::contracts {

  enum class assertion_kind : unspecified {
    pre = 1,
    post = 2,
    assert = 3
  };

  enum class evaluation_semantic : unspecified {
    ignore = 1,
```

```
    observe = 2,
    enforce = 3,
    quick_enforce = 4,
    prove = 5,
    diagnose = 6↩
  };

  enum class detection_mode : unspecified {
    predicate_false = 1,
    evaluation_exception = 2
  };

  class contract_violation {
    // no user-accessible constructor
  public:
    contract_violation(const contract_violation&) = delete;
    contract_violation& operator=(const contract_violation&) = delete;

    see below ~contract_violation();

    const char* comment() const noexcept;
    contracts::detection_mode detection_mode() const noexcept;
    bool is_terminating() const noexcept;
    assertion_kind kind() const noexcept;
    source_location location() const noexcept;
    evaluation_semantic semantic() const noexcept;
  };

  void invoke_default_contract_violation_handler(const contract_violation&);
}
```

17.10.2 **Enumerations** **[support.contract.enum]**

Modify section 17.10.2[support.contract.enum], Table 45 [support.contract.enum.semantic]:

Table 45 — Enum `evaluation_semantic` **[tab:support.contract.enum.semantic]**

Name	Meaning
`ignore`	Ignore evaluation semantic
`observe`	Observe evaluation semantic
`enforce`	Enforce evaluation semantic
`quick_enforce`	Quick-enforce evaluation semantic
`prove`	Prove evaluation semantic
`diagnose`	Diagnose evaluation semantic

6 Conclusion

We believe this is a proposal that fits cleanly into the model of C++26 Contracts and adds some utility. If others wish to continue to pursue such ideas, we strongly recommend that this be the approach they build upon.

Acknowledgments

Claude (Anthropic) was used for editorial assistance during the preparation of this paper, as well as significant parts of the prototype implementations.

Bibliography

[P3097R3] Timur Doumler, Joshua Berne, and Gašper Ažman, "Contracts for C++: Virtual functions", 2026
http://wg21.link/P3097R3

[P3100R9] Timur Doumler and Joshua Berne, "A framework for systematically addressing undefined behaviour in the C++ Standard", 2026
http://wg21.link/P3100R9

[P3400R5] Joshua Berne, "Controlling Contract-Assertion Properties", 2026
http://wg21.link/P3400R5

[P3595R1] Joshua Berne and Iain Sandoe, "Configuration of Contract Evaluation Semantics", 2026
http://wg21.link/P3595R1

[P3850R1] Timur Doumler and Joshua Berne, "A proposed plan for extending Contracts in C++29", 2026
http://wg21.link/P3850R1

[P4019R1] Jonas Persson, "constant_assert", 2026
http://wg21.link/P4019R1

[P4021R2] Jonathan Grant, "compile_assert - an assert that evaluates at compile time", 2026
http://wg21.link/P4021R2

[P4363R0] Jonathan Grant, "C++ Contracts compile-time evaluation semantic: static", 2026
http://wg21.link/P4363R0